EVALUACIÓN PRÁCTICA GRUPAL • DESARROLLO DE APLICACIONES MÓVILES

Consigna: Integración Arquitectónica y Persistencia Local

Ponderación: 20% | Modalidad: Grupal | Plazo de entrega: 26/10/2026 (prox. Lunes)

> **🎯 Propósito de la Evaluación**
> Consolidar la aplicación práctica de la arquitectura modular (SoC), el modelado de clases mediante Programación Orientada a Objetos (POO), el registro de fechas y timestamps, y la persistencia estructurada con JSON dentro del proyecto de aplicación móvil desarrollado por el equipo.

1. Requisitos Técnicos Obligatorios

La aplicación móvil desarrollada en KivyMD 2.0 debe cumplir estrictamente con los siguientes estándares técnicos:

- **Arquitectura Modular (Separación de Responsabilidades - SoC):** El proyecto debe estructurarse limpiamente en carpetas: views/ (pantallas y widgets KivyMD), services/ (lógica de negocio y archivos, 100% libre de Kivy), y models/ (clases POO de dominio). El archivo main.py actúa únicamente como orquestador.
- **Modelado POO Avanzado:** La entidad principal del proyecto (ej. Producto, Tarea, Paciente, Reserva) debe modelarse como una clase formal de Python que incluya un método to_dict() para serializar sus atributos.
- **Persistencia Estructurada con JSON:** Los datos creados en la App deben guardarse de forma permanente y recuperarse al reiniciar utilizando las funciones json.dump() y json.load().
- **Manejo Seguro de Ficheros:** Toda operación de lectura o escritura en disco debe implementarse obligatoriamente mediante el gestor de contexto with open(..., encoding='utf-8') para garantizar la seguridad de los datos.
- **Fechas, Horas y Timestamps:** Cada registro generado debe incorporar marcas de tiempo automáticas utilizando el módulo datetime o time, formateándose de manera legible para el usuario en la interfaz.
- **Librería Adicional del Stack Integrada:** El proyecto debe incorporar al menos una (1) de las librerías analíticas o científicas revisadas en el módulo (random, time con perf_counter(), numpy, scipy o matplotlib) cumpliendo una función real dentro de la App.

2. Entregable Documental: Informe Técnico Escrito

Cada grupo deberá entregar un informe técnico en formato PDF (máximo 5 páginas) que detalle la implementación de la arquitectura y la persistencia en su software. El documento debe contener la siguiente estructura:

1. **Portada y Definición del Proyecto.**

2. Mapeo de la Arquitectura (SoC): Esquema o descripción detallada de cómo distribuyeron las responsabilidades entre views/, services/ y models/.

3. Justificación del Modelo POO: Explicación de la clase principal desarrollada y cómo opera el patrón to_dict().

4. Implementación de Ficheros, Fechas y JSON: Fragmentos clave del código comentado, explicando el uso del bloque with y las funciones de persistencia JSON.

5. Integración de la Librería Adicional: Descripción técnica de la librería seleccionada (ej. numpy o matplotlib) y su aporte al flujo de la aplicación.

3. Dinámica de Evaluación: Demostración y Defensa Oral

El día de la entrega, cada equipo participará en una sesión presencial de 15 minutos dividida en dos etapas:

- **Demostración en Vivo (5 min):** El grupo ejecuta la aplicación (en emulador o dispositivo físico), demuestra la carga inicial desde el JSON, registra un nuevo elemento (evidenciando la fecha automática) y comprueba la actualización del archivo en disco.
- **Ronda de Preguntas Técnicas (10 min):** El docente realizará preguntas cruzadas a los integrantes para comprobar el dominio individual del código fuente y la arquitectura desarrollada.

4. Rúbrica de Evaluación

Criterio	Insuficiente (1 pt)	Aceptable (3 pts)	Excelente (5 pts)
Arquitectura Modular (SoC)	Código monolítico concentrado en main.py sin separación de carpetas ni capas.	Estructura organizada pero con pequeñas mezclas de lógica en las vistas o servicios.	Estructura impecable (views, services, models) con separación total de responsabilidades.
Persistencia JSON y POO	No implementa clases POO ni JSON; los datos se pierden al cerrar la aplicación.	Persistencia funcional pero almacena diccionarios planos sin clases POO formales.	Clases POO robustas con to_dict() y persistencia estructurada limpia con json.dump/load.
Manejo de Ficheros y Fechas	No manipula ficheros en disco ni registra marcas de tiempo o fechas.	Ficheros abiertos con open() sin with, o fechas estáticas/hardcodeadas en la app.	Uso seguro y correcto de with open() con codificación utf-8 y fechas/timestamps automáticos.
Librería Adicional Integrada	Sin rastros de ninguna librería analítica o científica adicional en el proyecto.	Librería implementada de forma forzada o solo con un print aislado en consola.	Librería adicional integrada limpiamente aportando valor real al flujo de la aplicación.
Informe Técnico y Defensa	No presentan informe y el grupo desconoce el funcionamiento del código.	Informe incompleto o disparidad notable en el dominio técnico entre los integrantes.	Informe técnico impecable y dominio total del código demostrado en la defensa oral.